Effects

Releases a lock on `*this` held by the current thread. If this is the last lock on `*this` held by the calling thread, any threads are blocked waiting to acquire a lock on `*this`. Unblocks one of them.

Postconditions

The number of locks on `*this` held by the calling thread is reduced by one.

Throws

Nothing.

D.5.3. std::timed_mutex class

The `std::timed_mutex` class provides support for locks with timeouts on top of the basic mutual exclusion and synchronization facility provided by `std::mutex`. Prior to accessing the data protected by the mutex, the mutex must be *locked* by calling `lock(), try_lock(), try_lock_for()`, or `try_lock_until()`. If a lock is already held by another thread, an attempt to acquire the lock will fail (`try_lock()`), block until the lock can be acquired (`lock()`), or block until the lock can be acquired or the lock attempt times out (`try_lock_for()` or `try_lock_until()`). Once a lock has been acquired (whichever function was used to acquire it), it must be released by calling `unlock()` before another thread can acquire the lock on the mutex.

`std::timed_mutex` meets the `TimedLockable` requirements.

Class definition

```
c"ass time"_mutex
{
pub"ic:
    time"_mutex(time"_mutex const&)="e"ete;
```